



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 12 – Token Standards and Token-Based DApps



Token Standards and Token-Based DApps



- 1. Token Standards in Ethereum**
- 2. ERC-20 Standard Concept and Example**
- 3. ERC-20 Token Interaction from Frontend**
- 4. ERC-721 Concept and Example**
- 5. Exercises**

1. Token Standards in Ethereum

Token: Programmable Digital Asset on Blockchain

Ether vs Token:

Ether	Token
Native currency of Ethereum	Created by smart contracts
Used to pay gas fees	Used in applications
Controlled by protocol	Controlled by contract logic

Examples:

- ❖ *Stablecoins (USDT, USDC)*
- ❖ *Game assets*
- ❖ *Reward points*
- ❖ *NFTs*

1. Token Standards in Ethereum

A token is:

- ❖ A smart contract that manages balances
- ❖ A digital asset represented by contract state
- ❖ Transferable between addresses

Problem:

- ❖ Every token contract could be different
- ❖ Wallets and exchanges cannot understand all tokens

Common Token Standards in Ethereum: Common interface rules

Standard	Token Type	Use Case
ERC-20	Fungible token	Currency, reward points
ERC-721	Non-fungible token (NFT)	Digital art, certificates
ERC-1155	Multi-token	Games

1. Token Standards in Ethereum

ERC-20 vs ERC-721

	ERC-20	ERC-721
Token type	Fungible	Non-fungible Token (NFT)
Balance	balanceOf(address)	ownerOf(tokenId)
Transfer	transfer(address, amount)	transferFrom(from, to, tokenId)
Use case	Money, points	Ownership, certificates



2. ERC-20 Standard Concept and Example

ERC-20 defines a **common interface** for fungible tokens.

Required functions:

- ❖ `totalSupply()`
- ❖ `balanceOf(address)`
- ❖ `transfer(address,uint256)`
- ❖ `approve(address,uint256)`
- ❖ `allowance(address,address)`
- ❖ `transferFrom(address,address,uint256)`

Events

- ❖ `Transfer(address from, address to, uint256 value)`
- ❖ `Approval(address owner, address spender, uint256 value)`

2. ERC-20 Standard Concept and Example

Sample Token:

- ❖ Token balances are state variables
- ❖ Transfers update contract state
- ❖ Events notify off-chain systems

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract SimpleToken {
    string public name = "MyToken";
    string public symbol = "MTK";
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    event Transfer(address indexed from, address indexed to, uint256 value);
    constructor(uint256 _supply) {
        totalSupply = _supply;
        balanceOf[msg.sender] = _supply;
    }
    function transfer(address to, uint256 amount) public returns (bool) {
        require(balanceOf[msg.sender] >= amount, "Insufficient balance");
        balanceOf[msg.sender] -= amount;
        balanceOf[to] += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }
}
```

2. ERC-20 Standard Concept and Example

Sample ERC-20 token with OpenZeppelin

OpenZeppelin is a professional blockchain library

- ❖ Widely used in industry
- ❖ Audited security code
- ❖ Standard implementations
- ❖ Modular design

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
contract MyToken is ERC20, Ownable {
    constructor() ERC20("MyToken", "MTK") Ownable(msg.sender) {
        _mint(msg.sender, 1000000 * 10 ** decimals());
    }
    function mint(address to, uint amount) public onlyOwner {
        _mint(to, amount);
    }
}
```

- ❖ Full ERC-20 compliance
- ❖ Ownership control
- ❖ Safe minting

2. ERC-20 Standard Concept and Example

Decimals Concept

Most ERC-20 tokens use 18 decimals.

Solidity does not support float numbers

```
function decimals() public pure returns (uint8) {  
    return 18;  
}
```

Example:

❖ *Raw balance: 1,000,000,000,000,000,000*

❖ *Decimals: 18*

❖ *Actual balance: 1 MTK*

=> Sending 0.5 MTK = sending the on-chain value of 500,000,000,000,000,000

3. ERC-20 Token Interaction from Frontend

Contract connection

```
import { ethers } from "https://cdn.jsdelivr.net/npm/ethers@6/dist/ethers.min.js";

const tokenAddress = "TOKEN_CONTRACT_ADDRESS_HERE";

const abi = [
  "function balanceOf(address owner) view returns (uint256)",
  "function transfer(address to, uint256 amount) returns (bool)"
];

const provider = new ethers.BrowserProvider(window.ethereum);
const signer = await provider.getSigner();
const tokenContract = new ethers.Contract(tokenAddress, abi, signer);
```

3. ERC-20 Token Interaction from Frontend

Read Token Balance

```
const address = await signer.getAddress();  
const balance = await tokenContract.balanceOf(address);  
console.log("Balance:", ethers.formatUnits(balance, 18));
```

Transfer Tokens

```
const tx = await tokenContract.transfer("RECEIVER_ADDRESS", ethers.parseUnits("10", 18));  
await tx.wait();  
console.log("Transfer completed!");
```

Listening to Transfer Events

```
tokenContract.on("Transfer", (from, to, value) => {  
    console.log(`Transfer: ${from} -> ${to}, amount: ${ethers.formatUnits(value, 18)}`);  
});
```

4. ERC-721 Standard Concept and Example

ERC-721 defines **non-fungible tokens** (NFTs).

- ❖ Each token has a unique ID
- ❖ Ownership is tracked individually

ERC-721 represents ownership, not balance.

Functions:

- ❖ `ownerOf(tokenId)`
- ❖ `safeTransferFrom(from, to, tokenId)`
- ❖ `approve(to, tokenId)`

4. ERC-721 Standard Concept and Example

Sample ERC-721 Token

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleNFT {
    mapping(uint256 => address) public ownerOf;
    uint256 public nextTokenId;
    event Transfer(address indexed from, address indexed to, uint256 indexed tokenId);
    function mint() public {
        uint256 tokenId = nextTokenId;
        ownerOf[tokenId] = msg.sender;
        nextTokenId++;
        emit Transfer(address(0), msg.sender, tokenId);
    }
}
```

4. ERC-721 Standard Concept and Example

Sample ERC-721 Token: using OpenZeppelin (Compliant)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";

contract MyNFT is ERC721 {
    uint public tokenId;

    constructor() ERC721("MyNFT", "MNFT") {
    }

    function mint() public {
        tokenId++;
        _mint(msg.sender, tokenId);
    }
}
```

5. Exercises

Ex 1: Build a custom ERC-20 token with:

- ❖ Name
- ❖ Symbol
- ❖ Total supply = 1,000,000 tokens.
- ❖ Assign all tokens to deployer
- ❖ Prevent transfers above 1,000 tokens per transaction.

Ex 2: Build a web token wallet page

- ❖ Connect MetaMask
- ❖ Show token balance of current user
- ❖ Transfer tokens to another address using frontend

5. Exercises

Ex 3: Build a Token Faucet DApp:

- ❖ Limit token claim frequency
- ❖ Show claim history
- ❖ Handle errors and invalid inputs

Ex 4: Build an NFT system:

- ❖ Users mint NFTs
- ❖ Each NFT has unique tokenId
- ❖ Frontend shows: tokenId/owner address
- ❖ One address can mint max 3 NFTs